



IMD1116

COMPUTAÇÃO DE

ALTO DESEMPENHO

MPI:

COMUNICAÇÃO NÃO BLOQUEANTE

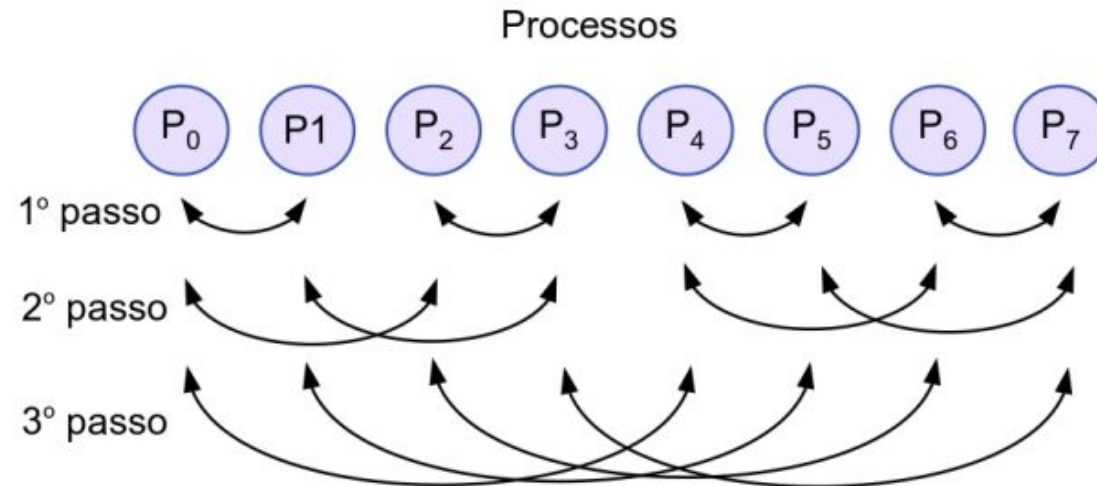
Prof. Samuel Xavier de Souza
Prof. Carla Santana

Algoritmo Recursive Doubling - MPI

Atv Extra - Algoritmo Recursive Doubling

Implemente, uma **versão usando comunicação não bloqueante** do algoritmo *Recursive doubling*.

Você tem até final da aula para implementar o programa e enviar pdf com o código via SIGAA.



PROGRAMAÇÃO PARALELA

- Objetivos deste tópico
 - Funções para comunicação não bloqueante

Comunicação Ponto-a-ponto não bloqueantes

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Request req_send, req_recv;
    MPI_Status status;
    // ...
    if (rank == 0) {
        strcpy(msg, "Mensagem nao bloqueante");
        MPI_Isend(msg, strlen(msg) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &req_send);
        printf("Processo 0 iniciou envio\n");
        // Simula computação enquanto o envio ocorre
        for (int i = 0; i < 3; i++) {
            printf("Processo 0 computando... %d\n", i);
            sleep(1);
        }
        // Garante que terminou
        MPI_Wait(&req_send, &status);
        printf("Processo 0 terminou envio\n");
    } else if (rank == 1) {
        MPI_Irecv(msg, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &req_recv);
        printf("Processo 1 iniciou recepcão\n");
        // Testa continuamente se a mensagem chegou
        while (!flag) {
            MPI_Test(&req_recv, &flag, &status);
            if (!flag) {
                printf("Processo 1 ainda esperando...\n");
                sleep(1); // simula trabalho
            }
        }
        printf("Processo 1 recebeu: %s\n", msg);
        // (opcional aqui, pois já completou)
        MPI_Wait(&req_recv, &status);
    }
    MPI_Finalize();
    return 0;
}
```

Comunicação Ponto-a-ponto não bloqueantes

- **MPI_Isend** : envia um mensagem
- **MPI_Irecv** : recebe uma mensagem.

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Request req_send, req_recv;
    MPI_Status status;
    // ...
    if (rank == 0) {
        strcpy(msg, "Mensagem nao bloqueante");
        MPI_Isend(msg, strlen(msg) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &req_send);
        printf("Processo 0 iniciou envio\n");
        // Simula computação enquanto o envio ocorre
        for (int i = 0; i < 3; i++) {
            printf("Processo 0 computando... %d\n", i);
            sleep(1);
        }
        // Garante que terminou
        MPI_Wait(&req_send, &status);
        printf("Processo 0 terminou envio\n");
    } else if (rank == 1) {
        MPI_Irecv(msg, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &req_recv);
        printf("Processo 1 iniciou recepcão\n");
        // Testa continuamente se a mensagem chegou
        while (!flag) {
            MPI_Test(&req_recv, &flag, &status);
            if (!flag) {
                printf("Processo 1 ainda esperando...\n");
                sleep(1); // simula trabalho
            }
        }
        printf("Processo 1 recebeu: %s\n", msg);
        // (opcional aqui, pois já completou)
        MPI_Wait(&req_recv, &status);
    }
    MPI_Finalize();
    return 0;
}
```

Comunicação Ponto-a-ponto não bloqueantes

- **MPI_Isend** : envia um mensagem
- **MPI_Irecv** : recebe uma mensagem.
- As operações não bloqueantes não retornam imediatamente

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Request req_send, req_recv;
    MPI_Status status;
    // ...
    if (rank == 0) {
        strcpy(msg, "Mensagem nao bloqueante");
        MPI_Isend(msg, strlen(msg) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &req_send);
        printf("Processo 0 iniciou envio\n");
        // Simula computação enquanto o envio ocorre
        for (int i = 0; i < 3; i++) {
            printf("Processo 0 computando... %d\n", i);
            sleep(1);
        }
        // Garante que terminou
        MPI_Wait(&req_send, &status);
        printf("Processo 0 terminou envio\n");
    } else if (rank == 1) {
        MPI_Irecv(msg, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &req_recv);
        printf("Processo 1 iniciou recepcao\n");
        // Testa continuamente se a mensagem chegou
        while (!flag) {
            MPI_Test(&req_recv, &flag, &status);
            if (!flag) {
                printf("Processo 1 ainda esperando...\n");
                sleep(1); // simula trabalho
            }
        }
        printf("Processo 1 recebeu: %s\n", msg);
        // (opcional aqui, pois já completou)
        MPI_Wait(&req_recv, &status);
    }
    MPI_Finalize();
    return 0;
}
```

Comunicação Ponto-a-ponto não bloqueantes

- **MPI_Isend** : envia um mensagem
- **MPI_Irecv** : recebe uma mensagem.
- As operações não bloqueantes não retornam imediatamente
- **MPI_Request** pode ser usado para testar se a operação foi finalizada.
- **MPI_Test** testa se a operação foi concluída
- **MPI_Wait** espera pela chegada real da mensagem.

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Request req_send, req_recv;
    MPI_Status status;
    // ...
    if (rank == 0) {
        strcpy(msg, "Mensagem nao bloqueante");
        MPI_Isend(msg, strlen(msg) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &req_send);
        printf("Processo 0 iniciou envio\n");
        // Simula computação enquanto o envio ocorre
        for (int i = 0; i < 3; i++) {
            printf("Processo 0 computando... %d\n", i);
            sleep(1);
        }
        // Garante que terminou
        MPI_Wait(&req_send, &status);
        printf("Processo 0 terminou envio\n");
    } else if (rank == 1) {
        MPI_Irecv(msg, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &req_recv);
        printf("Processo 1 iniciou recepcão\n");
        // Testa continuamente se a mensagem chegou
        while (!flag) {
            MPI_Test(&req_recv, &flag, &status);
            if (!flag) {
                printf("Processo 1 ainda esperando...\n");
                sleep(1); // simula trabalho
            }
        }
        printf("Processo 1 recebeu: %s\n", msg);
        // (opcional aqui, pois já completou)
        MPI_Wait(&req_recv, &status);
    }
    MPI_Finalize();
    return 0;
}
```

Comunicação Ponto-a-ponto não bloqueantes

- **MPI_Isend** : envia um mensagem
- **MPI_Irecv** : recebe uma mensagem.
- As operações não bloqueantes não retornam imediatamente
- **MPI_Request** pode ser usado para testar se a operação foi finalizada.
- **MPI_Test** testa se a operação foi concluída
- **MPI_Wait** espera pela chegada real da mensagem.

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Request req_send, req_recv;
    MPI_Status status;
    // ...
    if (rank == 0) {
        strcpy(msg, "Mensagem nao bloqueante");
        MPI_Isend(msg, strlen(msg) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &req_send);
        printf("Processo 0 iniciou envio\n");
        // Simula computação enquanto o envio ocorre
        for (int i = 0; i < 3; i++) {
            printf("Processo 0 computando... %d\n", i);
            sleep(1);
        }
        // Garante que terminou
        MPI_Wait(&req_send, &status);
        printf("Processo 0 terminou envio\n");
    } else if (rank == 1) {
        MPI_Irecv(msg, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &req_recv);
        printf("Processo 1 iniciou recepcão\n");
        // Testa continuamente se a mensagem chegou
        while (!flag) {
            MPI_Test(&req_recv, &flag, &status);
            if (!flag) {
                printf("Processo 1 ainda esperando...\n");
                sleep(1); // simula trabalho
            }
        }
        printf("Processo 1 recebeu: %s\n", msg);
        // (opcional aqui, pois já completou)
        MPI_Wait(&req_recv, &status);
    }
    MPI_Finalize();
    return 0;
}
```

MPI_Isend e MPI_Irecv

```
int MPI_Isend(void* mensagem, int cont, MPI_Datatype tipo_mpi, int destino, int etiq, MPI_Comm com, MPI_Request *pedido)
```

```
int MPI_Irecv(void* mensagem, int cont, MPI_Datatype tipo_mpi, int origem, int etiq, MPI_Comm com, MPI_Request *pedido)
```

- **pedido**: retorno imediato da função, um manipulador do tipo MPI_Request. Esse manipulador pode ser testado uma única vez (MPI_Test) e retornar, ou o processo pode car em espera ocupada (MPI_Wait) aguardando pela chegada real da mensagem.
-

MPI_Wait e MPI_Test

int MPI_Wait(MPI_Request *pedido, MPI_Status *estado)

int MPI_Test(MPI_Request *pedido, int *flag, MPI_Status *estado)

- **estado**: identificar a origem e a etiqueta da mensagem recebida
 - **flag**: inteira que retorna resultado do teste
-

MPI_Wait**

int MPI_Wait(MPI_Request *pedido, MPI_Status *estado)

int MPI_Waitall(int cont, MPI_Request vetor_de_pedidos[], MPI_Status vetor_de_estados[])

int MPI_Waitany(int cont, MPI_Request vetor_de_pedidos[], int *indice, MPI_Status *estado)

int MPI_Waitsome(int cont_entra, MPI_Request vetor_de_pedidos[], int *cont_saida,
int vetor_de_indices[], MPI_Status vetor_de_estados[])

- **indice**: qual requisição terminou primeiro
- **cont**: quantidade de requisições
- **MPI_Waitall**: espera por todas as mensagens correspondentes
- **MPI_Waitany**: espera por apenas uma das mensagens
- **MPI_Waitsome**: espera uma ou mais mensagens

MPI_Test**

```
int MPI_Test(MPI_Request *pedido, int *flag, MPI_Status *estado)
```

```
int MPI_Testall(int cont, MPI_Request vetor_de_pedidos[], int *flag, MPI_Status vetor_de_estados[])
```

```
int MPI_Testany(int cont, MPI_Request vetor_de_pedidos[], int *indice, int *ag, MPI_Status *estado)
```

```
int MPI_Testsome(int cont_entra, MPI_Request vetor_de_pedidos[], int *cont_saida,  
                 int vetor_de_indices[], MPI_Status vetor_de_estados[])
```

PROGRAMAÇÃO EM MEMÓRIA DISTRIBUÍDA

Tarefa 15:

Implemente uma simulação da difusão de calor em uma barra 1D, dividida entre dois ou mais processos MPI. Cada processo deve simular um trecho da barra com células extras para troca de bordas com vizinhos.

Implemente três versões: uma com **MPI_Send/ MPI_Recv** , outra com **MPI_Isend/ MPI_Irecv e MPI_Wait** , e uma terceira usando **MPI_Test** para atualizar os pontos internos enquanto aguarda a comunicação. Compare os tempos de execução e discuta os ganhos com sobreposição de comunicação e computação.

Referências

